

SAMOTA Replication — Final Comparison Report

30 independent runs per algorithm per benchmark, budget = 900 simulator evaluations/run (FF runs an extra sensitivity-analysis phase first: 1080 evals on ADAS1/ADAS2, 1170 on RR).

Algorithms: PF (NSGA3), RS (Random), FF (Focused), MERLOT (RL), SAMOTA, SAMOTA+SW (sliding window)

Benchmarks: ADAS1 (6D, 3 reqs), ADAS2 (6D, 6 reqs), RR (9D, 6 reqs, only 4 ever violated)

Data sources (two separate 30-run datasets, see Caveats §6): *results/* — full 6-algorithm dataset, used for failure counts, coverage-rate/speed, and cost. *results_diversity/* — re-run of FF/MERLOT/SAMOTA/SAMOTA+SW with per-evaluation trace logging, used only for diversity (PF's traces already existed in results/).

1. Effectiveness — violations found & requirement coverage

Mean violations/run ($\pm$ std), coverage = avg. fraction of that benchmark's requirements violated at least once in a run (partial credit), stats = Mann-Whitney U + Vargha-Delaney A vs SAMOTA. Full tables: *results/tables/summary_comparison.csv*, *results/tables/{BENCH}_violations_per_run.csv*.

Benchmark	Algorithm	Violations/run	Coverage	p-value	Effect (A_AB)	Winner
ADAS1	PF	35.0 $\pm$ 10.4	42.2%	0.000*	0.02 (large)	PF finds more, covers less
	RS	5.4 $\pm$ 2.4	42.2%	0.000*	0.96 (large)	SAMOTA
	FF	18.8 $\pm$ 8.8	45.6%	0.025*	0.33 (medium)	FF finds more
	MERLOT	19.0 $\pm$ 11.7	75.6%	0.105	0.38 (small)	not significant
	SAMOTA	14.0$\pm$5.2	100%	—	—	reference
	SAMOTA+SW	15.7$\pm$4.8	100%	0.150	0.39 (small)	not significant
ADAS2	PF	139.0 $\pm$ 36.9	72.8%	0.000*	0.02 (large)	PF finds far more
	RS	25.1 $\pm$ 7.3	68.9%	0.000*	0.99 (large)	SAMOTA
	FF	44.5 $\pm$ 24.1	66.1%	0.000*	0.84 (large)	SAMOTA
	MERLOT	81.5 $\pm$ 47.5	85.6%	0.796	0.52 (negligible)	not significant
	SAMOTA	74.6$\pm$17.6	92.2%	—	—	reference
	SAMOTA+SW	79.8$\pm$19.0	100%	0.403	0.44 (small)	not significant
RR	PF	16.9 $\pm$ 9.5	41.1%	0.001*	0.25 (large)	PF finds more
	RS	0.2 $\pm$ 0.6	3.3%	0.000*	1.00 (large)	SAMOTA far better
	FF	3.9 $\pm$ 3.7	31.7%	0.000*	0.82 (large)	SAMOTA
	MERLOT	9.0 $\pm$ 10.3	31.1%	0.130	0.61 (small)	not significant
	SAMOTA	9.7$\pm$5.8	50.6%	—	—	reference
	SAMOTA+SW	12.0$\pm$6.9	50.6%	0.179	0.40 (small)	not significant

* statistically significant ($p < 0.05$, Mann-Whitney U vs SAMOTA)

Talking point: PF/NSGA3 raw-optimizes the whole Pareto front for the entire budget, so it racks up more total violations, but at the cost of requirement coverage — it repeatedly hits the same easy requirement instead of spreading effort. SAMOTA deliberately reallocates budget away from already-covered objectives, trading violation count for broad coverage. This shows up directly: SAMOTA/SAMOTA+SW have the highest coverage % on every benchmark despite lower raw counts on ADAS1/ADAS2.

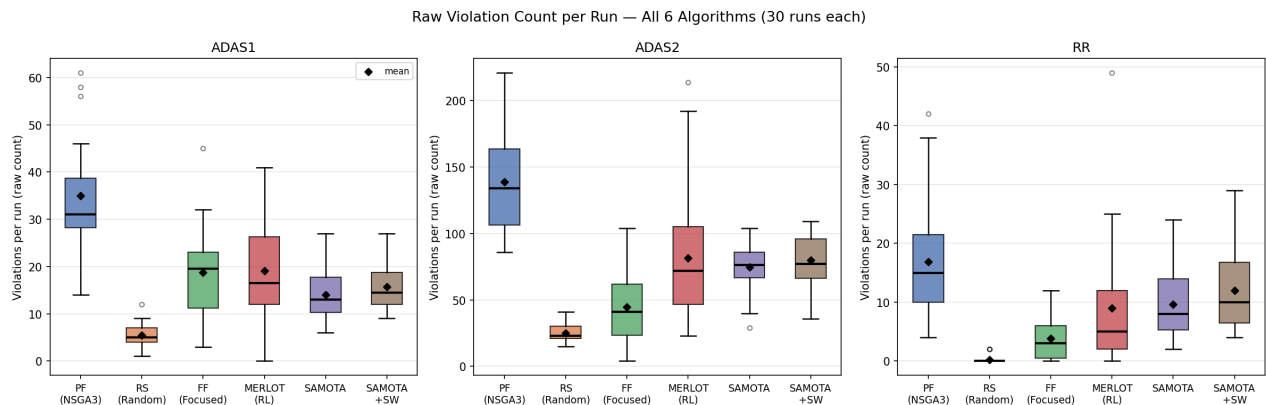


Figure 1a. Raw violation-count distribution (30 runs), all six algorithms including RS

2. Coverage rate & speed — % of runs that hit every violatable requirement

"Full coverage" = every requirement observed as violated by at least one algorithm was violated at least once within that specific run (binary per run, stricter than §1's partial-credit metric). RS excluded — no per-eval trace data available for it.

Benchmark	PF	FF	MERLOT	SAMOTA	SAMOTA+SW
ADAS1 rate	13.3%	16.7%	63.3%	100%	100%
ADAS1 median speed	61.2% budget	84.6%	43.8%	38.1%	37.5%
ADAS2 rate	16.7%	6.7%	63.3%	76.7%	100%
ADAS2 median speed	23.7% budget	106.8% (never within budget)	42.7%	45.8%	42.6%
RR rate	3.3%	3.3%	0%	23.3%	16.7%
RR median speed	71.7% budget	48.2%	N/A	74.9%	58.1%

Talking point: SAMOTA/SAMOTA+SW are the only algorithms that reliably cover every requirement on ADAS1/ADAS2. RR is hard for everyone — MERLOT never fully covers it in 30 runs, and even SAMOTA only reaches 23%.

ADAS1 — Coverage Rate & Speed (budget=900, 3 violatable reqs)

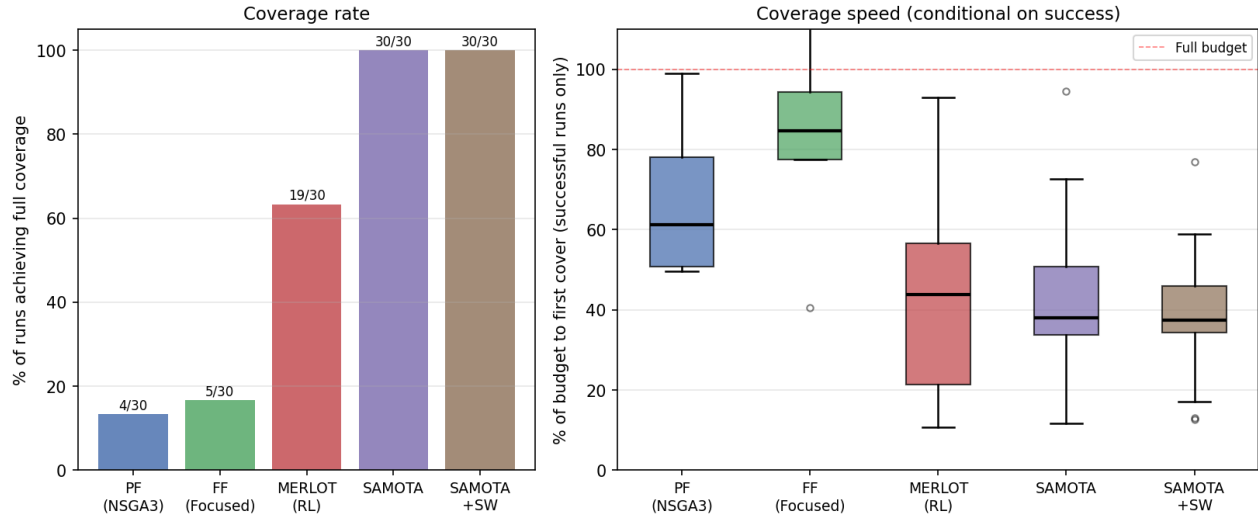


Figure 2a. ADAS1 — coverage rate (left) and speed (right)

ADAS2 — Coverage Rate & Speed (budget=900, 6 violatable reqs)

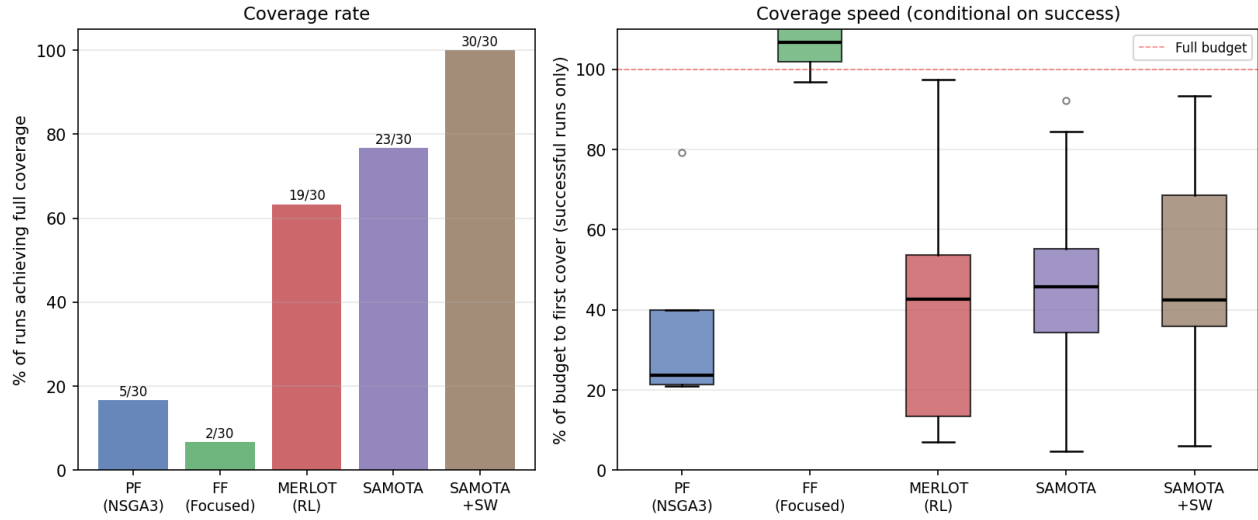


Figure 2b. ADAS2 — coverage rate (left) and speed (right)

RR — Coverage Rate & Speed (budget=900, 4 violatable reqs)

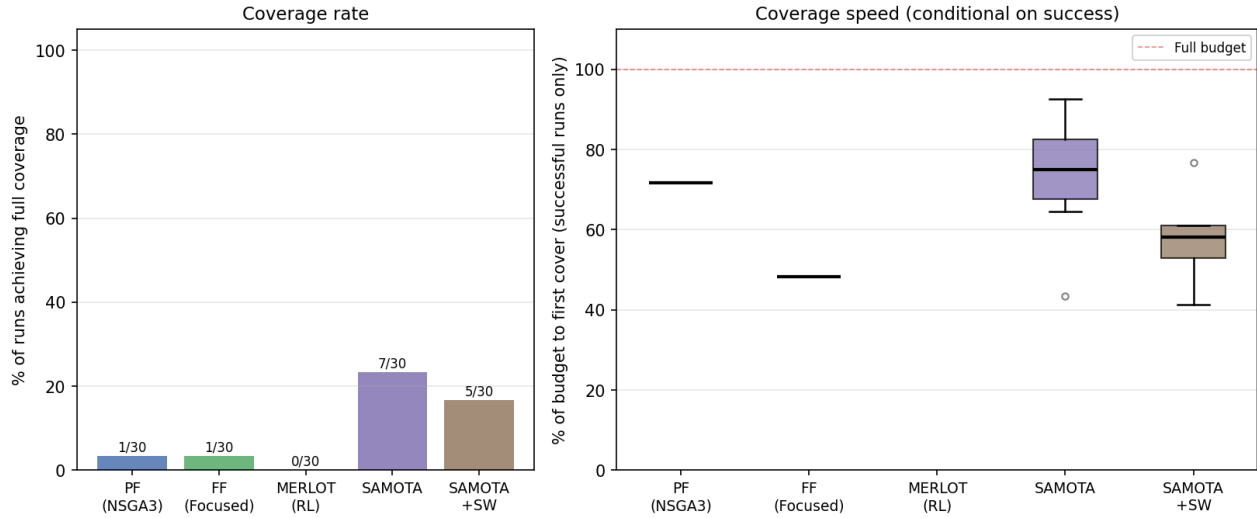


Figure 2c. RR — coverage rate (left) and speed (right)

2.1 Area Under Coverage Curve (AUCC) — normalized summary metric

RS excluded (no per-eval trace data). For each run, build a step curve of "fraction of reachable requirements violated at least once so far" over % of that run's own budget used, then AUCC = mean of that curve over the whole run (1.0 = covered everything immediately, 0.0 = never covered anything). Unlike the binary full-coverage rate above, AUCC gives partial credit for runs that cover some — but not all — reachable requirements, and rewards covering them earlier. Computed from real per-evaluation trace data, 30 runs each.

Benchmark	PF	FF	MERLOT	SAMOTA	SAMOTA+SW
ADAS1 AUCC	0.311±0.094	0.298±0.120	0.580±0.268	0.643±0.145	0.708±0.100
ADAS2 AUCC	0.584±0.118	0.511±0.091	0.697±0.210	0.762±0.106	0.759±0.080
RR AUCC	0.274±0.080	0.268±0.162	0.234±0.164	0.252±0.118	0.264±0.106

Talking point: AUCC confirms §2's full-coverage-rate finding on ADAS1/ADAS2 — SAMOTA/SAMOTA+SW lead by a wide margin, covering reachable requirements earlier and more consistently across the whole run, not just eventually. RR is the interesting case: all five algorithms cluster tightly (0.234–0.274) and SAMOTA+SW edges out plain SAMOTA on AUCC (0.264 vs 0.252) even though §2 showed plain SAMOTA with the higher *binary* full-coverage rate (23.3% vs 16.7%). This is consistent with §5's finding that SAMOTA+SW makes more partial progress on RR (finds more, more diverse violations) but is less likely to close out every last reachable requirement within budget — AUCC rewards the former, the binary rate only credits the latter.

Coverage-of-Reachable-Requirements Over Run Progress (mean ± std, 30 runs)

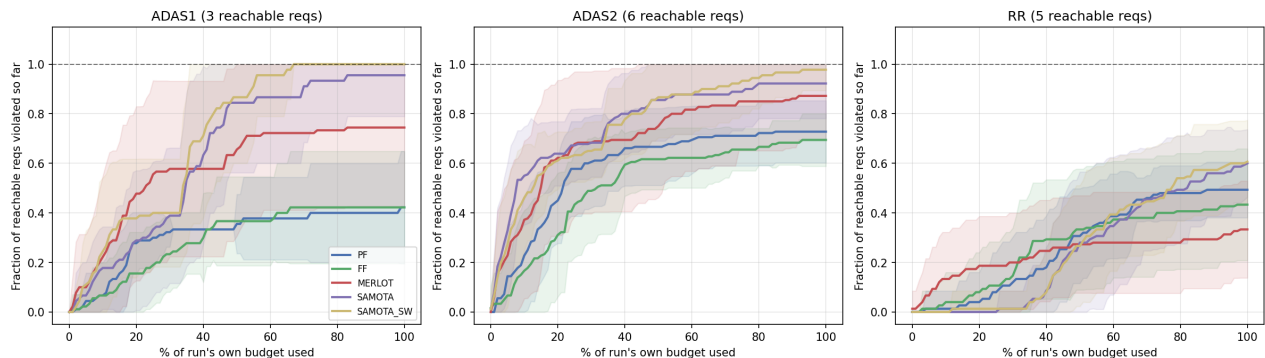


Figure 2d. Coverage-of-reachable-requirements over % of run's own budget used (mean $\pm$ std, 30 runs)

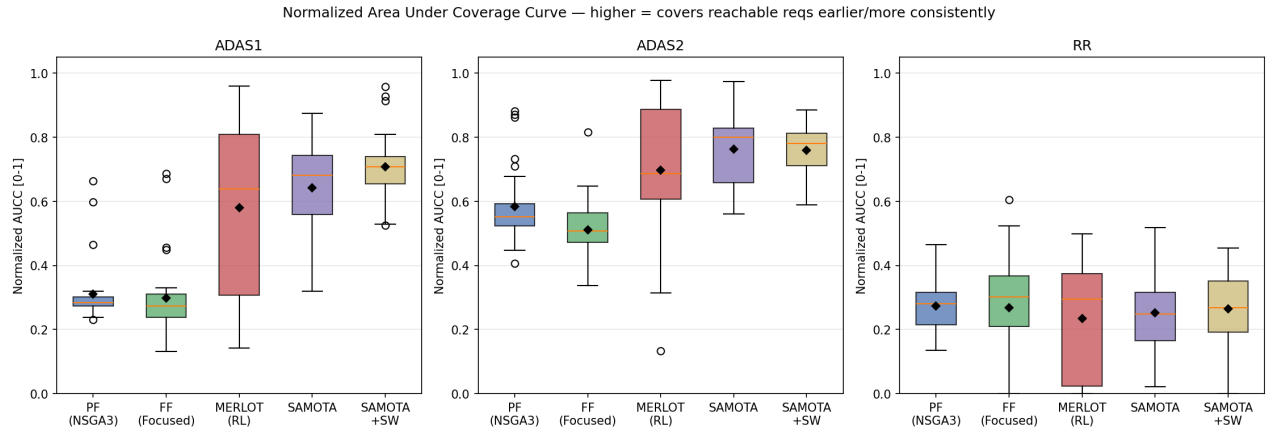


Figure 2e. Normalized AUCC distribution per algorithm, one panel per benchmark

3. Cost — wall-clock time

Real measured average minutes/run (parsed from run.log "Total Duration" lines for PF/RS/FF/MERLOT, from meta_SAMOTA_30.csv elapsed_s for SAMOTA/SAMOTA+SW — no estimation).

Benchmark	PF	RS	FF	MERLOT	SAMOTA	SAMOTA+SW
ADAS1	43.6 min	49.6	67.5	49.7	103.2	80.0
ADAS2	44.3 min	56.0	60.8	45.5	103.2	85.3
RR	27.1 min	32.9	24.6	17.0	39.1	40.9

SAMOTA/SAMOTA+SW are 1.5–4x slower per run (surrogate-model training overhead) than the lighter-weight baselines. But per-run cost alone is misleading if a "cheap" algorithm has to be restarted repeatedly to ever succeed. **Expected time to first full coverage** (geometric model: accounts for restart-on-failure using each algorithm's own success rate):

$$E[T_{success}] = \left(\frac{1}{p} - 1\right) mean_fail_time + mean_success_time$$

p = success rate (fraction of the 30 runs that reach full coverage within the 900-eval budget). **mean_fail_time** / **mean_success_time** = average wall-clock time among the failed / successful runs respectively (§3's per-run table above). Intuition: $(1/p - 1)$ is the expected number of *extra* restarts needed before one succeeds, each restart costing on average mean_fail_time minutes wasted, plus the mean_success_time minutes of the run that finally lands. As $p \rightarrow 0$ this $\rightarrow \infty$ (never reliably succeeds); as $p \rightarrow 1$ it collapses to plain mean_success_time.

Benchmark	PF	FF	MERLOT	SAMOTA	SAMOTA+S W
ADAS1	313 min	393	50.5	42.6	31.4
ADAS2	238 min	916	43.1	79.6	42.7
RR	807 min	724	∞ (never succeeds)	157	228

Talking point: once you account for the fact that PF/FF succeed rarely and have to be restarted, SAMOTA/SAMOTA+SW are actually the cheapest path to full coverage on ADAS1/ADAS2 by a wide margin, despite the highest per-run cost. RR is the exception — plain SAMOTA beats SAMOTA+SW there (see §5).

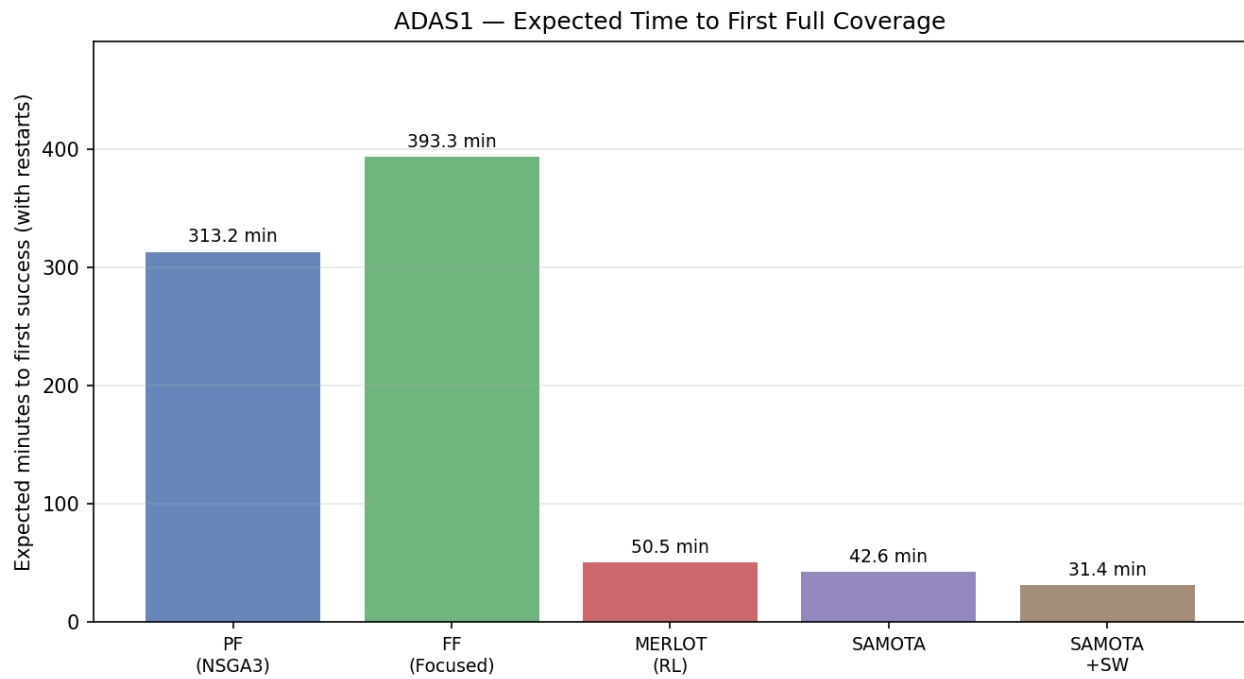


Figure 3a. ADAS1 — expected minutes to first full coverage (with restarts)

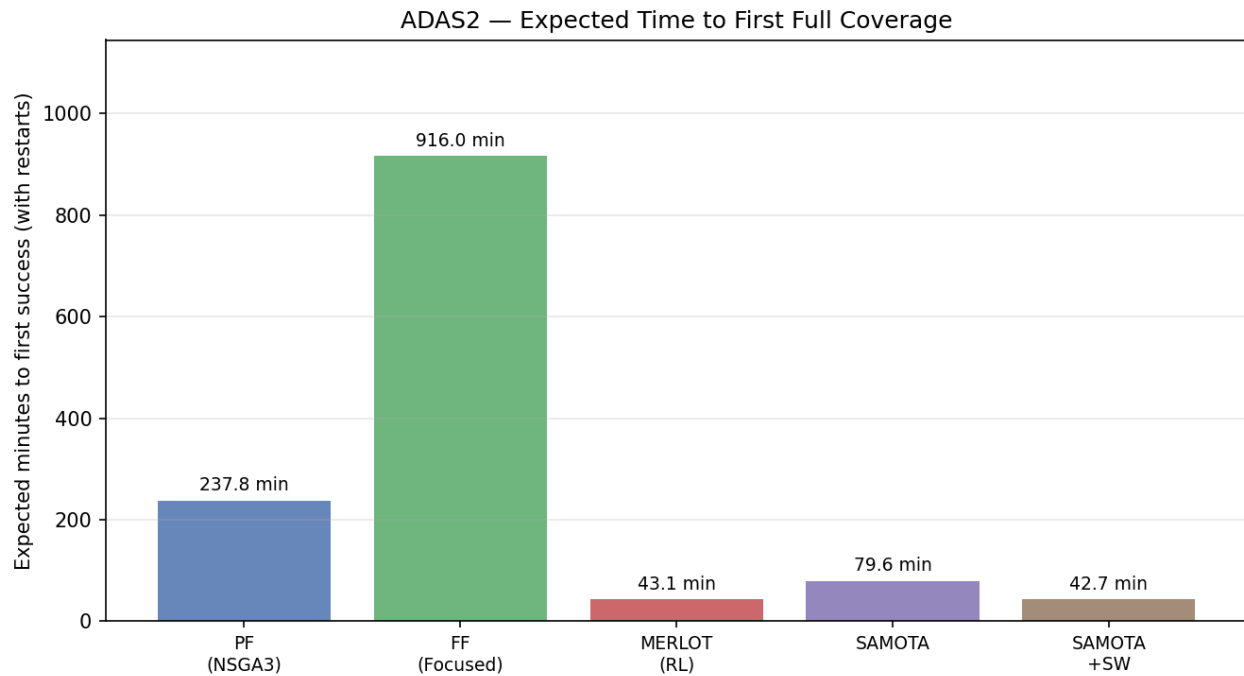


Figure 3b. ADAS2 — expected minutes to first full coverage (with restarts)

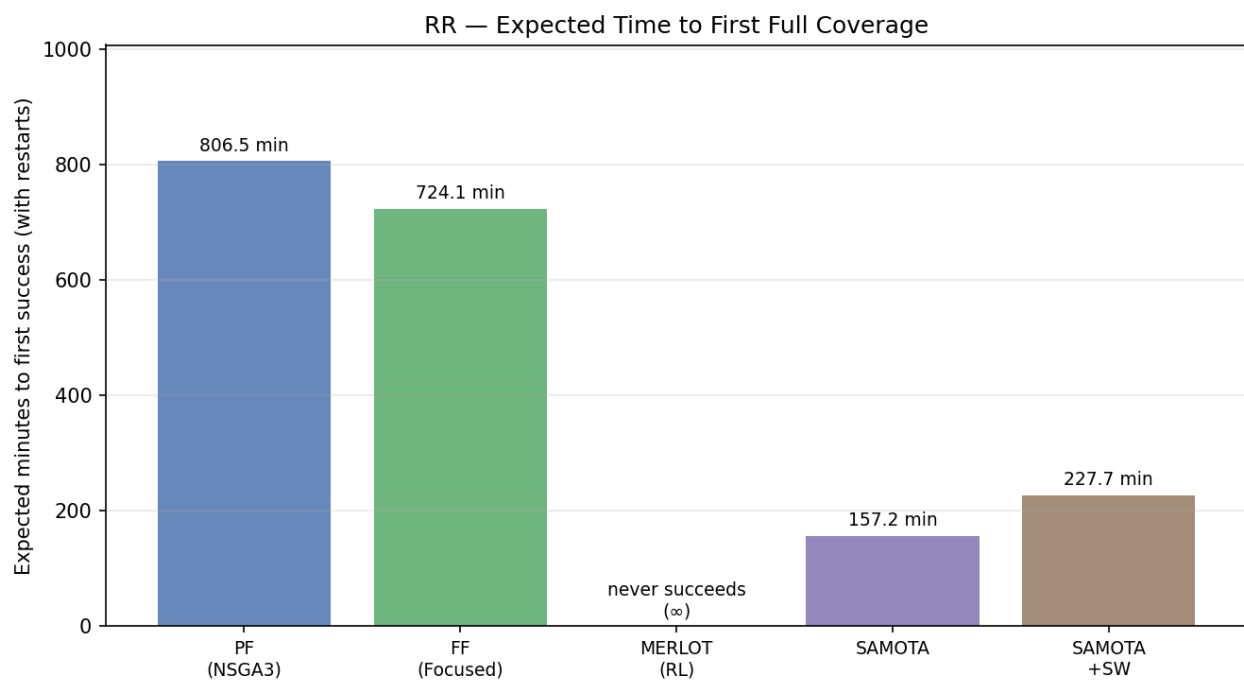


Figure 3c. RR — expected minutes to first full coverage (with restarts)

4. Diversity of failures

Parameter-space APD (avg pairwise Euclidean distance between failing test-case parameter vectors, MinMax-normalized) and Hamming APD (avg pairwise distance between violation patterns), computed on real per-evaluation trace data, 30 runs each.

Benchmark	Metric	PF	FF	MERLOT	SAMOTA	SAMOTA+SW
ADAS1	param APD	0.80	0.90	0.92	1.20	1.21
ADAS1	Hamming APD	0.009	0.018	0.199	0.340	0.337
ADAS2	param APD	0.80	0.99	1.01	1.18	1.22
ADAS2	Hamming APD	0.147	0.196	0.224	0.275	0.284
RR	param APD	0.55	0.63	0.68	1.16	1.30
RR	Hamming APD	0.050	0.092	0.008	0.156	0.177

Talking point: SAMOTA/SAMOTA+SW consistently produce the most diverse failures on every benchmark, both in parameter space and in which combination of requirements gets violated — this is a direct consequence of the budget-reallocation behavior noted in §1. SAMOTA+SW edges out plain SAMOTA on diversity everywhere, even on RR where it loses on coverage rate. Note: MERLOT's RR Hamming APD (0.008) is an outlier — nearly every RR failure it finds hits the same violation pattern, unlike its ADAS1/ADAS2 behavior (0.20–0.22).

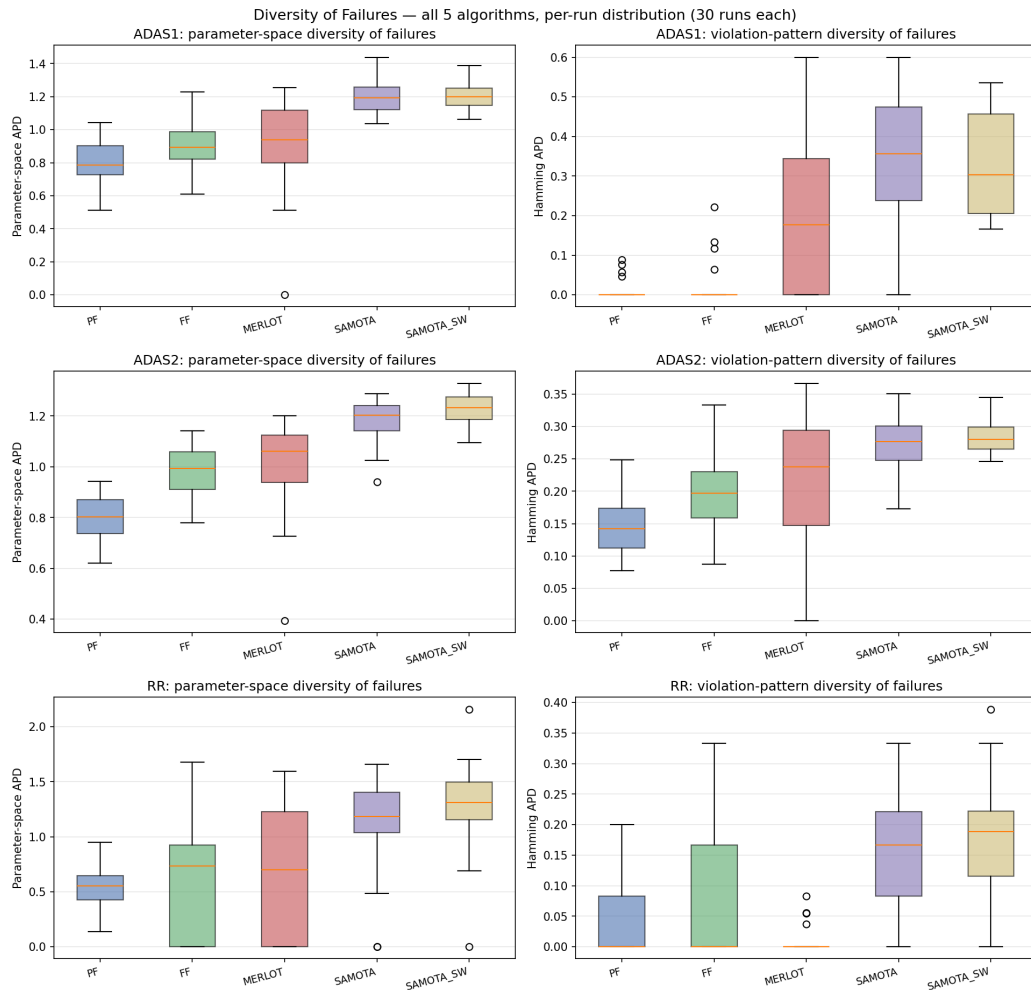


Figure 4a. Parameter-space APD and Hamming APD, all 5 algorithms x 3 benchmarks

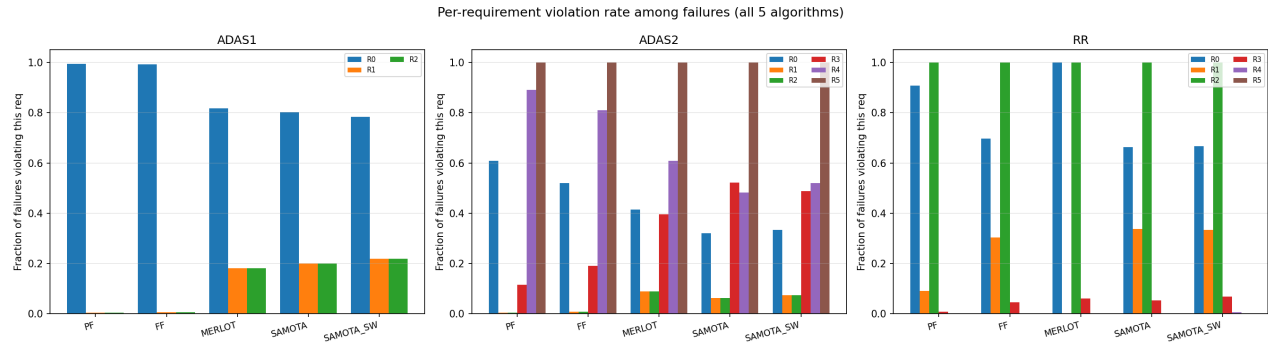


Figure 4b. Per-requirement violation rate by algorithm and benchmark

5. SAMOTA vs SAMOTA+SW (sliding window) — direct comparison

5.1 How the sliding window works

Plain SAMOTA retrains its GS/LS surrogate models on every evaluation collected so far in the run. SAMOTA+SW instead retrains only on the most recent **window_size** evaluations, discarding older samples once that cap is passed:

$$X_{train} = X[-w :] \text{ if } n > w, \text{ else } X \quad (w = \text{window_size} = 150)$$

Value used: **window_size = 150**, fixed across all three benchmarks and both dimensionalities (6D for ADAS1/ADAS2, 9D for RR). This is a flat CLI default (`--window_size 150` in `run_all_experiments.py`) — there is no derivation or tuning sweep behind that number in the codebase or commit history; it was not scaled to budget (900) or to variable count. That's the root of the RR regression discussed below: the same 150-sample cap that works well at 6D covers a much sparser fraction of a 9D input space.

5.2 Direct comparison

Benchmark	Time/run	Violations/run	Full coverage rate	Diversity (param APD)	Expected time to success
ADAS1	SW 22% faster (80 vs 103 min)	SW higher (15.7 vs 14.0)	tied (100%)	SW slightly higher	SW 26% faster (31 vs 43 min)
ADAS2	SW 17% faster (85 vs 103 min)	SW higher (79.8 vs 74.6)	SW higher (100% vs 76.7%)	SW higher	SW ~2x faster (43 vs 80 min)
RR	SW slightly slower (41 vs 39 min)	SW higher (12.0 vs 9.7)	SW lower (16.7% vs 23.3%)	SW higher	SW slower (228 vs 157 min)

Conclusion: SAMOTA+SW is a clear win on ADAS1 and ADAS2 — faster per run, finds more violations, matches or beats coverage rate, more diverse, and reaches full coverage roughly 2x faster on ADAS2. RR is the one benchmark where the sliding window hurts full-coverage reliability, even though it still finds more (and more diverse) violations there.

Likely explanation: RR is 9-dimensional vs ADAS's 6D. A fixed window_size=150 retains a constant amount of training history regardless of dimensionality — in a higher-dimensional space that same window covers a sparser sample of the input space, so the surrogate models the local region less accurately near the end of a run, right when GS/LS most needs precision to close out the last uncovered requirement.

Suggested improvements (not yet implemented):

1. **Scale window size with dimensionality** — e.g. `window_size = k * n_variables` instead of a flat constant, so RR's surrogate gets a comparably dense sample to ADAS1/ADAS2's.
2. **Adaptive window size** — grow the window when surrogate error/hit-rate is degrading instead of using a fixed cutoff.
3. (Data-quality, not SW-specific) PFES_SAMOTA.py's Adaptive Random Testing phase calls `np.random.uniform()` without ever calling `np.random.seed()`, so runs aren't reproducible under a fixed `--seed` even though `run_seed` is computed and passed everywhere else. Doesn't affect the conclusions above (all comparisons are 30-run aggregate statistics), but worth fixing before any follow-up work that expects bit-for-bit reproducibility.

6. Caveats (for Q&A)

- §1/§2/§3 use results/; §2.1/§4 use results_diversity/ (a separate 30-run instrumented rerun, used both for AUCC and for diversity — PF's traces already existed in results/). Per-run numbers are not expected to match 1:1 between the two datasets — the pipeline has a confirmed non-determinism source (see suggestion #3 above) — but each dataset is internally consistent

(30 independent runs) and each metric is computed from the dataset built for it.

- §2's "coverage rate" (all-or-nothing per run) and §1's "coverage" (partial-credit average) are different metrics — don't conflate the two numbers for the same algorithm. §2.1's AUCC is a third, continuous metric (partial credit *and* rewards earlier coverage) — see §2.1 for how it can diverge from §2's binary rate.
- RS is excluded from §2/§2.1/§4 (coverage-speed, AUCC, and diversity) because it has no per-evaluation trace files in this dataset, only run-level aggregates.